

FINAL PLAN — Smart Market Watchlist (Groww Code 2026)

This is the single source of truth. Ignore earlier back-and-forth — build from this.

1. Required features (from the problem statement — must have all of these)

1. **Create and manage a watchlist** — add/remove stocks.
2. **View latest market information** — live-ish price, % change, per stock.
3. **Return later and see what has changed** — this is the core ask. You must persist *what the user last saw* and diff it against current data.

Everything else in the brief ("what counts as meaningful change," "how state persists," "how you handle stale data," "how it scales") are **design decisions you make and justify**, not extra features to build. You answer them through how you build the 3 things above.

2. Your definition of "meaningful change" (say this explicitly in your pitch)

A change is meaningful if it's **large relative to that stock's own normal daily movement**, not a fixed % for every stock. Concretely:

- % move since the user's last visit (not since market open)
- compared against that stock's own recent average daily move
- flagged if it crosses $\sim 1.5x$ that average

This is one line of math, not ML — and explicitly choosing a simple rule over ML here is a *strength* to point out, not a gap.

3. Additional features (build in this order, stop whenever time runs out)

#	Feature	Effort	Why it impresses
1	Freshness badge ("updated 12s ago" / "showing last known price")	Low	Directly answers "handle stale/delayed data"
2	Fallback to last cached price if live fetch fails	Low	Same — real resilience, not just a UI label
3	Highlighted/flagged rows for meaningful changes	Low	Core product interpretation
4	"Digest" summary card ("3 things changed since Tuesday")	Medium	Product thinking, Simple + Delightful
5	AI-generated one-line explanation for a flagged stock	Medium	Your one AI touchpoint (see below)
6	Live deployment (Vercel + Render)	Low–Medium	Judges can try it without cloning code
7	Multiple watchlists	Medium	Nice-to-have only if time allows

Do NOT build: price prediction, sentiment scoring, chatbots, gamification. These read as unfocused or irresponsible for a fintech context and the FAQ explicitly warns against building what you *think* judges want.

4. Where AI goes — exactly one place

AI is NOT used for: detecting meaningful change (that's the z-score/threshold math), fetching data, or anything the app depends on to function.

AI IS used for: turning a flagged change into a natural sentence.

- Input: structured data you already computed, e.g. `{symbol: "NVDA", change: 4.8%, volumeRatio: 2.1x}`
- Output: one sentence describing the pattern — e.g. "Up 4.8% on more than double the usual volume."
- **Rule: never let it invent a cause** (like "due to earnings") unless you're pulling a real headline to ground it in. Guessing causes in a financial app is a real trust/safety risk — call this restraint out explicitly in your pitch, it's a stronger answer than a flashier feature.
- **Always have a non-AI fallback** (plain string template) if the API call fails or times out — this is both good engineering and doubles as another resilience story.

Implementation: one backend endpoint (`POST /api/explain`), Claude or Gemini API, cached per change-event so you're not re-calling it every refresh.

5. Real-time data — how to actually implement it

- **Library:** `yahoo-finance2` (npm package, no API key, works for NSE tickers like `RELIANCE.NS`).
 - **Where it runs:** only in the **backend** — the frontend never calls it directly.
 - **Pattern:** backend polls each watched symbol every 20–30 seconds, stores the latest price + timestamp in Supabase (`price_snapshots` table), and serves that to the frontend. The frontend polls *your backend* every 20–30s (not the stock API directly) — this is both simpler and is what lets you show a clean "freshness" timestamp.
 - **Resilience:** wrap every fetch in try/catch; on failure, serve the last known snapshot and mark it stale in the response. This single pattern is what answers "how do you handle stale/delayed/conflicting data" — build it early, not as a bolt-on.
-

5b. Cross-device persistence (currently missing — add this)

"Return later and see what changed" only counts as solved if it works from *any* device, not just the browser you built it in. That requires real identity, not just local state.

- Use **Supabase Auth** — simplest option: magic-link email login (no password flow to build) or even anonymous auth (Supabase creates a stable user ID without a signup form) if you want zero friction for the demo.
- Every watchlist row and `price_snapshots`/"last seen" row is tied to that `user_id` in Postgres — so logging in on a second device pulls the same data. This is what actually answers "how state persists across sessions/devices," not just "we used a database."
- Keep it minimal: you don't need profile pages or settings — just enough auth to prove identity persists.

5c. Scaling for larger watchlists / more users (say this even if you don't fully build it)

You won't have time to load-test this, but you need a real answer, not a guess:

- **Dedupe fetches across users:** if 500 users all watch NVDA, don't fetch NVDA 500 times. Your backend should poll each *unique symbol* once (e.g. maintain a set of all symbols currently watched by anyone) and let every user's view read from that one shared `price_snapshots` row. This turns your fetch cost from $O(\text{users})$ to $O(\text{unique symbols})$ — a real scaling decision, not hand-waving.
- **Batch the API calls:** `yahoo-finance2` supports fetching multiple symbols in one call — use that instead of one request per symbol.
- **Index your Postgres tables** on `(user_id, symbol)` so lookups stay fast as rows grow.
- State this explicitly in your pitch even if your live demo only has a handful of symbols — judges are grading the *design decision*, not whether you stress-tested it.

5d. Race conditions / conflicting data (explicit "Edge Cases & Resilience" ask)

Pick one concrete scenario and actually handle it — don't leave this vague:

- **Same watchlist edited from two devices at once** (e.g. add AAPL on phone, remove it on laptop seconds later): use a simple `updated_at` timestamp per row and last-write-wins, or let Postgres's own row-level locking handle it — either is fine, just pick one and be able to explain it.
- **Two data sources disagree** (if you added a backup API): show the primary source's value but note in your README that you detected a mismatch, or simply always trust one source and document why.

Add one line about this to your README — this maps directly to "Edge Cases & Resilience" in the judging table (see mapping below).

6. Final tech stack (locked in — do not change again)

Layer	Choice	Why
Frontend	React + Vite (separate app/folder)	Fast setup, component model fits real-time UI, judges see it as its own deliverable
Backend	Node.js + Express (separate app/folder)	Same language as frontend = no context-switching under time pressure; owns all data logic
Database	Supabase (managed Postgres)	Zero server setup, free, backend-only access
Stock data	yahoo-finance2 npm package	Free, no key, called only from backend
AI (one endpoint only)	Claude or Gemini API	Explains flagged changes; always has a fallback
Editor	Kiro (or VS Code)	Kiro's spec-first flow is fine, just don't let it slow down the sprint — keep specs small and per-feature

7. Time-boxed order of work (adjust to your remaining hours)

1. Confirm frontend + backend both run independently, connected to Supabase.
2. Backend: watchlist CRUD (add/remove/list stocks) → test with Postman.
3. Backend: price fetch route with fallback-to-cache on failure.
4. Backend: "meaningful change" diff logic against last-seen snapshot.
5. Frontend: watchlist UI — list, add stock, price, % change, freshness badge, highlighted flagged rows.
6. Only if time remains: digest card, AI explain endpoint, deployment.
7. Deliberately break the price fetch (turn off wifi briefly) and confirm the fallback UI works — this is your best "resilience" demo moment.
8. Write README (setup instructions) + 100-word pitch covering: your change definition, one edge case handled, where AI was/wasn't used.

9. Push to GitHub, submit.

8. What your pitch/README must mention (this is what's graded)

- Your definition of "meaningful change" and why it's relative, not fixed.
- One concrete edge case you handled (API down, stale data, etc.).
- Where you used AI and — just as important — where you deliberately didn't.
- One honest trade-off from the time constraint.

9. Direct mapping to the judging table (use this to self-check before submitting)

Judging dimension	Where it's covered in this plan
Engineering Depth (architecture, correctness, reliability, scalability)	Separate frontend/backend/DB architecture (§6), fallback-to-cache on fetch failure (§5), symbol-dedupe scaling approach (§5c)
Product & Problem Interpretation (understanding beyond the obvious brief)	Relative "meaningful change" definition (§2), digest card, freshness badges — not just a plain ticker list
Edge Cases & Resilience (failures, race conditions, integrity, unreliable dependencies)	Fetch fallback (§5), cross-device race-condition handling (§5d), deliberate "break the wifi" test (§7 step 7)
Code Quality & Simplicity (maintainable, not over-engineered)	Simple threshold math instead of ML for change detection (§2), AI used in exactly one scoped place (§4)
Originality & Thoughtfulness (independent, considered choices)	Every "you decide" item in the brief (meaningful change, persistence, staleness, scaling, simple-vs-complex) has an explicit, stated decision in this document — write these into your pitch, don't leave them implicit

If your final submission can point to a specific line in this table for every row, you're aligned with the full brief — not just the three minimum features.